

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**Проектиране и разработване на уеб базирана
система за електронна търговия с компютърна и
офис техника**

ДИПЛОМНА РАБОТА

Изготвил:

Кирил Вълков, фак. № 121222194

Специалност: Компютърно и софтуерно инженерство

Научен ръководител:

ас. маг. инж. Деян Дънешки

София, 2026

СЪДЪРЖАНИЕ

Увод	1
1. Въведение в уеб-базирани платформи за онлайн търговия на компютърни части	2
2. Проектиране на уеб-базираната платформа	7
3. Програмна реализация	17
4. Ръководство за използване	26
Заклучение	39
Използвана литература	40

УВОД

Електронната търговия е сред най-динамично развиващите се области на съвременните информационни системи. Днешните онлайн магазини са комплексни приложения, които съчетават идентификация на потребители, представяне на продукти, управление на наличности, обработка на поръчки и устойчиво съхранение на данни. Предметната област на настоящата работа е продажба на компютърна и офис техника – продукти, масово търсени от индивидуални и фирмени клиенти, и предполагащи структуриране по категории, проследяване на наличности и асоцииране с доставчици.

Основната цел на дипломната работа е да се проектира и разработи уеб базирана система за електронна търговия с компютърна и офис техника, която поддържа регистрация на потребители, управление на фирми и продукти, разглеждане на каталог, работа с количка, финализиране на поръчка и проследяване на история. За постигането ѝ са формулирани задачите: анализ на предметната област, избор на технологичен стек, проектиране на многослойна архитектура, моделиране на домейн обектите, реализация на регистрация и сесийно управление, изграждане на каталог, реализация на checkout и история, базова локализация и примерен REST интерфейс.

Проектът е реализиран като Java уеб приложение върху Spring Boot 2.6.2 и Java 11. Уеб слойът използва Spring MVC и Thymeleaf, persistence слойът – Spring Data JPA и MySQL, а валидацията и сигурността – Hibernate Validator, ModelMapper и Spring Security Crypto чрез Pbkdf2PasswordEncoder. Обхватът включва back-end логиката, HTML интерфейс, MySQL persistence и Maven build. Извън него остават реална платежна интеграция, Spring Security, Docker и разширено автоматизирано тестване.

Изложението е организирано в четири глави. Първата въвежда контекста на уеб-базираните платформи и теоретичните основи. Втората представя проектирането – изисквания, архитектура и домейн модел. Третата описва програмната реализация на back-end, представящия слой и операционната среда. Четвъртата представя ръководство за използване и верификация. Заключението обобщава постигнатото, оценява решението и очертава насоки за бъдещо развитие.

1. ВЪВЕДЕНИЕ В УЕБ-БАЗИРАНИ ПЛАТФОРМИ ЗА ОНЛАЙН ТЪРГОВИЯ НА КОМПЮТЪРНИ ЧАСТИ

Настоящата глава поставя разработката в контекста на съществуващите подходи и теоретичните основи на уеб-базираните платформи за електронна търговия. Първо се прави преглед на типичните решения в областта и се обосновава изборът на самостоятелна разработка. След това се въвеждат основните теоретични понятия, върху които стъпва архитектурата на системата.

1.1. Преглед на съществуващите решения

Съвременните системи за електронна търговия се разделят на три основни групи: SaaS платформи с управлявана инфраструктура, self-hosted решения с готова функционалност и custom приложения върху общи уеб рамки. Изборът между тях зависи от необходимата гъвкавост, бюджет, срокове и налични технически компетенции. Настоящият проект попада в третата група и следва да бъде разглеждан като контролирана инженерна реализация, а не като конфигуриране на готов продукт.

Сред готовите платформи се открояват три типични представителя. Shopify [1] предлага цялостна SaaS инфраструктура с готов административен панел и интеграции, но ограничава архитектурния контрол и не е подходящ за задълбочен инженерен анализ. WooCommerce [2] е разширение за WordPress с ниска бариера за вход, но архитектурата му остава зависима от структурата на WordPress и плъгините. Magento Open Source [3] е мощна self-hosted платформа за по-сложни сценарии, чиято сложност често е непропорционална на учебните цели.

Таблица 1. Съпоставка между типични подходи за изграждане на електронен магазин

Подход	Предимства	Ограничения	Подходящост за дипломна разработка
Shopify	Бърз старт, хоствана инфраструктура	Ограничен архитектурен контрол	Ниска
WooCommerce	Лесно внедряване, богата екосистема	Зависимост от WordPress	Средна
Magento	Богата функционалност, разширяемост	Висока сложност	Средна към ниска
Custom Spring приложение	Пълен контрол върху модел и архитектура	По-голяма първоначална разработка	Висока

Таблица 1 показва, че custom Spring приложение е най-подходящият избор, когато целта е демонстрация на архитектурна последователност и проследимост на решенията от UI до базата данни. В Java екосистемата Spring Boot [4] осигурява стандартизиран модел за конфигурация и автоматична инициализация. Spring MVC [5] предлага естествен модел за маршрутизация и интеграция с Thymeleaf [6], а Spring Data JPA [7] позволява домейн обектите да бъдат описани като entity класове, без значителна ръчна SQL логика.

По отношение на архитектурния модел могат да се разграничат два доминиращи подхода: сървърно рендирано приложение и клиентски ориентирано приложение с REST/GraphQL API. Сървърно рендираният модел намалява общата сложност, тъй като представящият слой и бизнес логиката остават в една технологична среда, и позволява бърза реализация на форми и навигационни потоци. Клиентски ориентираният модел дава по-голяма динамичност, но изисква отделна front-end архитектура и по-сложен модел за сигурност. Настоящият проект избира доминиращо сървърно рендиран подход с допълнителна REST крайна точка, която демонстрира, че двата стила могат да съществуват в едно и също приложение.

Така позиционирано, приложението не се конкурира с пълнофункционални търговски платформи, а демонстрира как чрез добре подбрани технологии могат да бъдат реализирани основните механизми на електронен магазин в прозрачна и аналитично обясним форма. Подходящият избор зависи от целите: за бързо въвеждане в експлоатация SaaS решенията са по-силни, но за архитектурна прозрачност и образователна стойност custom Spring приложение е по-подходящо.

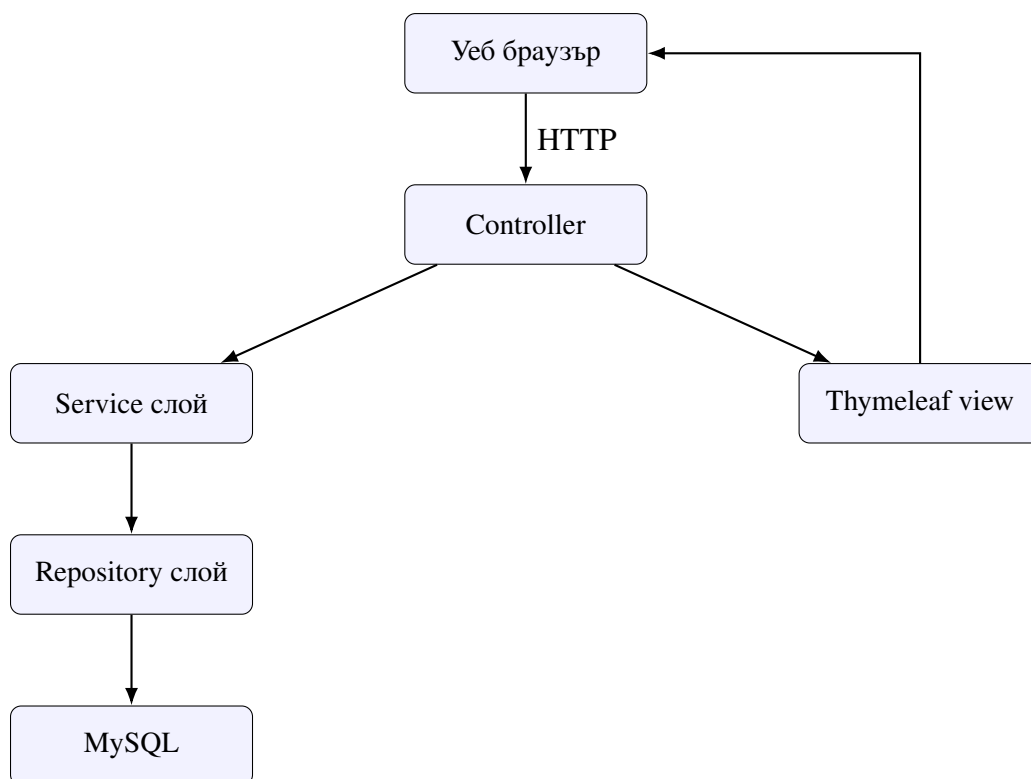
1.2. Теоретични основи

Архитектурата на разглежданата система стъпва върху няколко утвърдени концепции от уеб инженерството и софтуерната архитектура: клиент–сървърно взаимодействие, многослойна организация, MVC модел, обектно-релационно моделиране, валидация и сесийно управление. Този раздел въвежда именно онези идеи, които са пряко необходими за разбиране на по-нататъшното описание.

В основата стои клиент–сървърният модел: браузърът инициира HTTP заявка, сървърът прилага бизнес логика, работи с данните и връща отговор – HTML, пренасочване или JSON. Тъй като HTTP е безсъстояниен, приложението въвежда собствен механизъм за проследяване на потребителския контекст. В настоящия проект тази роля се поема от `HttpSession`, в която се съхранява електронната поща на удостоверения потребител. Този модел е лесен за разбиране и достатъчен за малка учебна система.

Многослойната архитектура [8, 9] разделя отговорностите в системата на слоеве с относително ясни граници – визуален слой, приложен слой, бизнес слой и слой за достъп до данни. Предимството е, че промени в един слой могат да бъдат извършвани с по-малко странични ефекти върху останалите. В настоящия проект многослойността е реализирана чрез контролери, услуги, repository интерфейси и шаблони, което позволява лесно проследяване на потока на данните.

Model–View–Controller [5] е класическият архитектурен шаблон за уеб приложения. В Spring MVC моделът представя данните за визуализация, view е шаблонът, а контролерът обработва заявката и избира коя view да бъде върната. Важно е, че терминът „модел“ в Spring MVC не съвпада с домейн модела: домейн обектите са entity класове, а Model в контролерите е контейнер за атрибути за Thymeleaf.



Фигура 1. Обобщен поток на обработка при сървърно рендирана заявка

Dependency injection е друга основна идея в Spring [4] – класовете получават зависимостите си отвън, вместо да ги създават ръчно. Това намалява свързаността и улеснява тестването. В настоящия проект контролерите получават услугите си чрез конструкторна инжекция, а конфигурационният клас дефинира bean инстанции за `Mapper` и `PasswordEncoder`.

Обектно-релационното съответствие [7] свързва обектния свят с релационния. В разглежданата система класове като `User`, `Company`, `Product`, `ShoppingCartEntity` и `HistoryEntity` са маркирани като JPA entity обекти и описват релациите в домейна. Важно е разграничението между домейн модел и модели за пренос на данни: за входни форми се използват binding модели, за вътрешен трансфер – service модели, а за изход – view модели. Това ограничава директното изтичане на persistence структурата към UI.

Валидацията на входни данни е фундаментална характеристика на всяка надеждна система. В Spring приложенията тя се реализира чрез Java Bean Validation анотации и обработка на `BindingResult` [10]. В конкретния проект ограниченията са дефинирани основно в binding моделите чрез анотации като `@Size`, `@NotBlank` и `@Positive`. Управлението на потребителска идентичност изисква паролите никога да не се съхраняват в явен вид, а да бъдат хеширани чрез устойчив механизъм [11, 12]; в проекта тази роля

изпълнява `Pbkdf2PasswordEncoder`, а сесийното удостоверяване е реализирано чрез `HttpSession`.

Представящият слой използва `Thymeleaf` [6] като шаблонен механизъм, интегриращ HTML маркировка с изрази за данни. Той позволява условна визуализация, итериране, двупосочно свързване на форми и локализирани съобщения. Локализацията в проекта е реализирана чрез `message bundles` и `interceptor` за смяна на езика, поддържайки английски и български вариант. Освен HTML слоя приложението предоставя и примерен REST интерфейс за максималната цена на продукт, който показва, че същата бизнес логика може да бъде представена както чрез HTML, така и чрез JSON, в съзвучие с принципите на ресурсно ориентираното представяне [13].

Така разгледаните понятия очертават теоретичния фундамент на разработката. Клиент-сървърният модел определя взаимодействието, многослойната архитектура организира отговорностите, MVC структурира уеб потока, ORM свързва домейн обектите с базата, а валидацията и хеширането осигуряват елементарна сигурност. Тази рамка е пряко материализирана в структурата на проекта, разгледана в следващата глава.

2. ПРОЕКТИРАНЕ НА УЕБ-БАЗИРАНАТА ПЛАТФОРМА

Настоящата глава представя проектирането на разработваната уеб-базирана платформа. След теоретичната рамка от предходната глава, тук фокусът се пренасочва към конкретните инженерни решения: изискванията към системата, нейната архитектурна организация и домейн моделът, който описва предметната област.

2.1. Анализ на изискванията и потребителските сценарии

Изискванията формират критерия, спрямо който се оценяват архитектурата, структурата на кода и степента, в която приложението изпълнява предназначението си. В случая на онлайн магазин за компютърна и офис техника те се формират както от типичните процеси в електронната търговия, така и от конкретния начин, по който проектът е структуриран в сорс кода.

2.1.1. Участници и потребителски цели

В системата могат да бъдат разграничени три типа участници. **Посетител без активна сесия** достъпва началната страница, регистрацията и входа, но няма право на достъп до защитените страници; логиката е реализирана чрез проверка за атрибут `email` в сесията. **Потвърден регистриран потребител** е основният активен участник – след потвърждение по електронна поща и успешен вход получава достъп до каталога, профила, фирмите, формите за добавяне, количката и историята. Липсва отделна роля от тип администратор, което е приемливо за учебен сценарий. **Външни инфраструктурни компоненти** включват MySQL базата за съхранение и SMTP сървър за изпращане на потвърждаващи писма.

Основните потребителски цели включват: създаване на акаунт и потвърждение по електронна поща; удостоверяване и работа в персонална сесия; въвеждане на фирмена информация; въвеждане на продукти с категория, цена, количество и изображение; разглеждане на каталог; добавяне на продукт в количка; финализиране на поръчка с адрес; и преглед на история. Тези цели са практическият еквивалент на функционалните изисквания.

2.1.2. Функционални и нефункционални изисквания

Функционалните изисквания на системата са обобщени в таблица 2. Те следват логична последователност от регистрация и вход към управление на данни, покупка и проследяване на резултата.

Таблица 2. Обобщение на основните функционални изисквания

ID	Изискване	Реализация в проекта
FR-1	Регистрация на потребител	Форма, запис в базата, хеширане на парола
FR-2	Потвърждение на акаунт	Имейл с линк и поле <code>isConfirmed</code>
FR-3	Вход и изход	Проверка на парола и управление на <code>HttpSession</code>
FR-4	Редакция на профил	Форма за профил и проверка на стара парола
FR-5	Добавяне на фирма	Контролер и <code>binding</code> модел за фирмени данни
FR-6	Добавяне на продукт	Категории, фирма, количество, цена, URL
FR-7	Разглеждане на каталог	Категорийни маршрути и Thymeleaf визуализация
FR-8	Добавяне в количка	Детайлен изглед, количество, проверка за наличност
FR-9	Финализиране на поръчка	Адрес, обща цена, създаване на история
FR-10	История на поръчките	Екран с записи и операция за изчистване
FR-11	Локализация	<code>messages.properties</code> , <code>messages_bg.properties</code>
FR-12	REST услуга	Крайна точка за максимална цена на продукт

Нефункционалните изисквания обхващат поддържаемост (ясно разделение по пакети и слоеве), четим и последователен потребителски интерфейс, коректност на

входните данни чрез валидация, устойчиво съхранение чрез MySQL и Hibernate, елементарна сигурност (хеширани пароли, ограничен достъп до защитени страници) и разширяемост, която позволява добавяне на нови категории, екрани и услуги без пренаписване на основата.

2.1.3. Ключови потребителски сценарии и ограничения

Функционалните изисквания се групират в няколко основни сценария. При **регистрация и първи вход** посетителят попълва име, фамилия, имейл и парола, системата създава потребител, хешира паролата и изпраща линк за потвърждение; след активиране потребителят влиза в системата. При **добавяне на фирма и продукт** потребителят въвежда фирма, после преминава към формата за нов продукт, избира категория, фирма, цена, количество и URL. При **разглеждане на каталог и покупка** потребителят филтрира продуктите, отваря детайлен екран, добавя желано количество в количката, въвежда адрес и потвърждава поръчката; системата създава исторически запис и изчиства количката. При **преглед на история и профил** потребителят може да редактира лични данни и да преглежда или изчиства записите на поръчките.

Анализът разкрива и характерни ограничения: липсата на отделни роли означава, че системата не разграничава административни от потребителски действия; локализацията е частична и не обхваща всички текстове; някои операции, които в production среда биха били защитени, тук са реализирани с по-прост модел. Тези ограничения не обезсмислят проекта – те очертават границата между учебна система и комерсиална платформа.

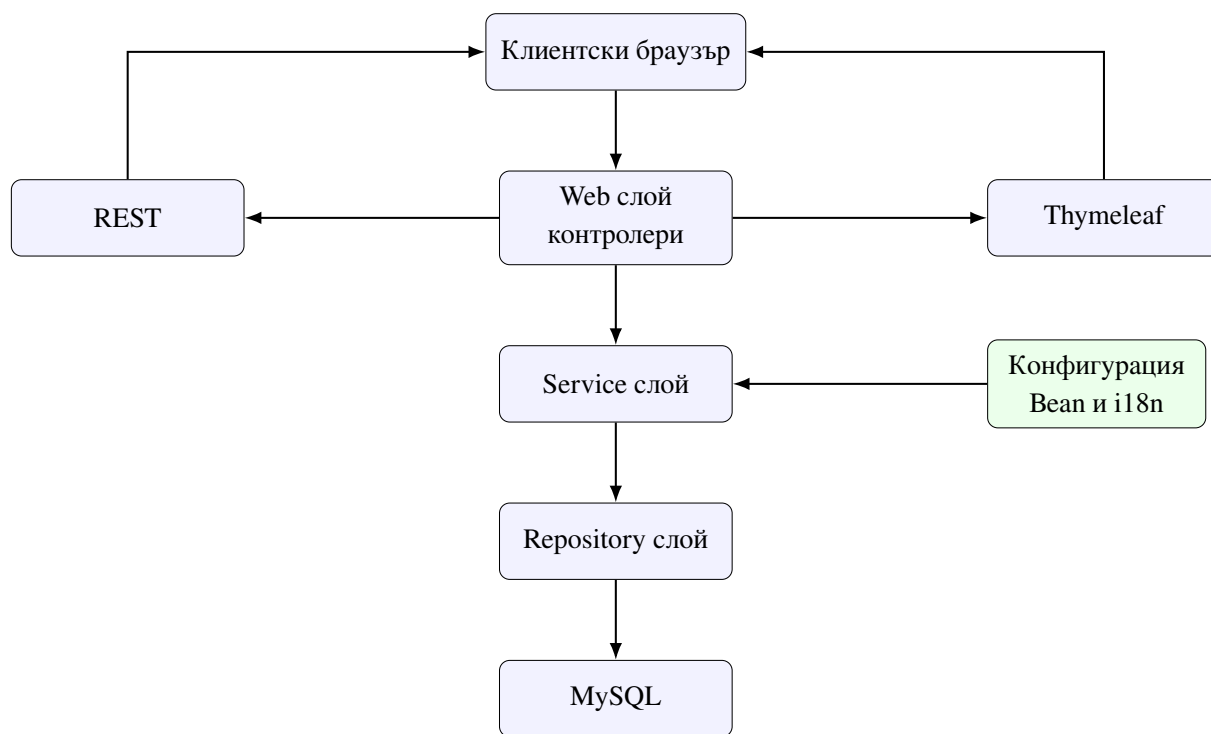
2.2. Архитектура на системата

Архитектурата е гръбнакът на системата, защото определя как отговорностите са разпределени, как се движат данните и до каква степен проектът може да се поддържа и развива. В настоящата разработка архитектурният избор е насочен към ясно разделение между уеб слой, бизнес логика, достъп до данни, конфигурация и визуализация.

2.2.1. Обща архитектурна постановка

Системата е реализирана като монолитно уеб приложение върху Spring Boot. Терминът „монолитно“ тук означава, че всички основни компоненти – уеб маршрутизация,

бизнес логика, persistence слой, HTML шаблони и конфигурация – се намират в един проект и се стартират в един процес. Това е подходящо, защото домейнът е компактен, учебната цел е прозрачност, а Spring Boot предоставя достатъчна организационна рамка за вътрешно добре структурирано приложение.



Фигура 2. Високо ниво на архитектурата

Фигура 2 показва, че приложението следва централен уеб поток, но може да предоставя резултати както чрез HTML визуализация, така и чрез JSON, което е белег за архитектурна гъвкавост.

2.2.2. Пакетна организация и входна точка

Основният Java код е разположен в `src/main/java/com/example/diplomen` проект и е разделен на под-пакети, които изразяват архитектурното разделение на отговорностите.

Таблица 3. Пакети в Java слоя и тяхното предназначение

Пакет	Основно предназначение
<code>config</code>	Конфигурационни класове за bean и локализация
<code>models.entities</code>	JPA entity обекти на домейна
<code>models.bindingModel</code>	Модели за приемане и валидация на входни форми
<code>models.serviceModel</code>	Междинни модели за пренос на данни
<code>models.viewModel</code>	Модели за визуализация в шаблоните или API
<code>repositories</code>	Spring Data JPA интерфейси за достъп до базата
<code>services & services.impl</code>	Service интерфейси и тяхната имплементация
<code>web</code>	MVC и REST контролери

Входната точка е класът `DiplomenProektApplication`, маркиран с `@SpringBootApplication`. Той не съдържа бизнес логика, а изпълнява единствено ролята на стартова точка, което предотвратява смесването на инфраструктурна и домейн логика още на най-високо ниво.

2.2.3. Слоевете на приложението

Конфигурационният слой включва `ApplicationBeanConfiguration` (дефинира bean компоненти за `ModelMapper` и `PasswordEncoder`) и `InitialConfig` (настройва `LocaleResolver` и `LocaleChangeInterceptor`). Централизираната конфигурация позволява подмяна на инфраструктурни избори на едно място.

Уеб слой съдържа контролерите `HomeController`, `UserController`, `CompanyController`, `ProductController` и `MaxPriceRestController`, разделени по функционални области. Контролерите запазват ролята си на координатори: валидират достъпа, проверяват binding резултата, извикват услуга и определят коя view да върнат – следват добрата практика контролерът да бъде „тънък“.

Service слой е организиран чрез интерфейси и имплементации, което

разграничава договора от реализацията и улеснява бъдещо тестване. Той обхваща потребителски операции (регистрация, вход, потвърждение, профил, изчисляване на цена), управление на фирми, управление на продукти, количка и история. Именно тук се вземат решения за последователността на операциите – например при checkout се координира обновяването на количката, историята и продуктите.

Persistence слойът е реализиран чрез Spring Data JPA repository интерфейси – UserRepository, CompanyRepository, ProductRepository, ShoppingCartRepository и HistoryRepository. Методи като findByEmail, getAllByUser_Id и deleteAllByUser_Id показват типичния Spring Data подход, при който заявките се изразяват чрез имената на методите. Repository слойът е правилно ограничен до операции по извличане, запис и изтриване, без самостоятелна бизнес логика.

Представящият слой се намира в src/main/resources/templates и src/main/resources/static. Шаблоните отразяват основните потребителски екрани, а статичните ресурси включват CSS, JavaScript и изображения. Слойът остава относително изолиран от persistence логиката и работи с вече подготвени атрибути.

2.2.4. Модели за трансфер на данни и архитектурен поток

Архитектурата използва четири типа модели: entity модели за persistence, binding модели за входни форми, service модели за междинен трансфер и view модели за визуализация. Този подход ограничава директната зависимост между базата и UI – например шаблоните могат да работят с view модели вместо с пълни entity обекти. ModelMapper намалява ръчните преобразувания.

Архитектурните потоци могат да бъдат илюстрирани с три ключови операции. При **регистрация** браузърът изпраща POST към потребителския контролер; той приема binding модел и предава към услугата, която проверява за дублиран имейл, създава entity, хешира паролата, записва чрез repository и изпраща потвърждаващ имейл. При **добавяне на продукт** контролерът валидира формата и предава към ProductService, която намира фирмата, създава нов продукт, преобразува категорията към CategoryEnum и записва обекта. При **checkout** контролерът приема адрес и обща цена и извиква HistoryService, която създава запис в историята, асоциира го с потребителя, изчиства количката и премахва продукти с нулева наличност – операция, която не може да бъде сведена до една repository заявка.

2.2.5. Силни страни и компромиси

Силните страни на архитектурата включват: ясна пакетна организация; разграничение между контролери, услуги и repository интерфейси; използване на отделни модели за различни етапи на обмен; централизация на инфраструктурни компоненти; наличие както на HTML, така и на REST интерфейс. Същевременно са налице компромиси: част от проверките за достъп са в контролерите, вместо в централизирана security архитектура; някои операции показват силна зависимост от конкретната структура на данните; архитектурата е обвързана с конкретна реализация на сесиен достъп и локална конфигурация. Това е разумно за учебен сценарий, но следва да бъде адресирано при бъдещо развитие.

2.3. Домейн модел и модел на данните

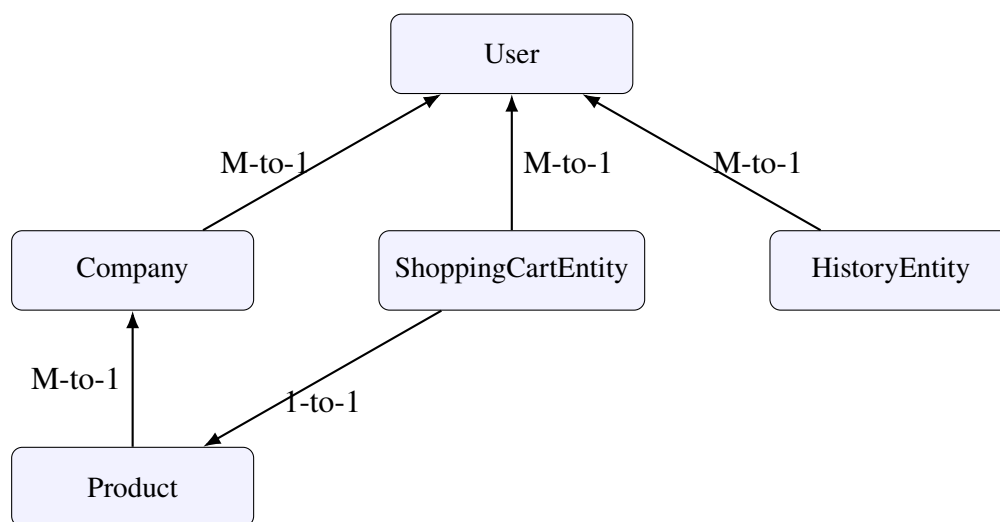
Домейн моделът описва основните участници и артефакти в процеса на електронна търговия: потребител, фирма, продукт, количка и история на поръчки. Макар приложението да не реализира сложна enterprise семантика, моделът е достатъчно богат, за да наложи ясно картографиране към релационна база.

2.3.1. Основни домейн понятия и entity модел

В предметната област могат да бъдат идентифицирани следните базови понятия. **Потребител** – физическо лице, което се регистрира, влиза, поддържа профил, добавя фирми и продукти, използва количка и има история. **Фирма** – търговска организация, асоциирана с конкретен потребител и използвана при добавяне на продукти. **Продукт** – каталоген артикул с име, цена, наличност, изображение и категория. **Запис в количка** – желание за покупка на определен продукт в дадено количество. **Исторически запис** – обобщена информация за приключена поръчка. Тези понятия образуват ядрото на домейна [14].

Persistence слойът използва JPA entity класове, които наследяват BaseEntity за централизиране на идентификатора. Класът User е основният aggregate обект – съдържа име, фамилия, парола, признак за потвърждение и електронна поща, и поддържа връзки към фирмите, количката и историята. Полето за електронна поща е уникално и има regex валидация. Класът Company съдържа име, mol, vat, адрес и признак за регистрация; принадлежи на потребител чрез @ManyToOne и има множество продукти чрез @OneToMany

oMany. Класът Product съдържа име, цена, количество, URL и категория (изброим тип CategoryEnum), с валидация @Min(0) за цена и количество. Класът ShoppingCartEntity моделира преходно състояние с три атрибута – желано количество, продукт (@OneToOne) и потребител (@ManyToOne). Класът HistoryEntity представя обобщен запис за реализирана поръчка с адрес, дата, цена и връзка към потребител – умишлено опростен модел, който не включва отделни позиции или статуси, но е достатъчен за учебно-приложен контекст.



Фигура 3. Обобщена схема на връзките между основните entity обекти

Фигура 3 показва, че потребителят е централният домейн елемент, към който се асоциират фирмите, количката и историята, а продуктите са свързани с фирмите и се използват в количката.

2.3.2. Категории, binding и view модели

Категоризационният модел е реализиран чрез CategoryEnum със стойности като VIDEOCARDS, PROCESSORS, HEADPHONES, PRINTERS, OFFICE_CHAIRS, SCANNERS, ROUTERS, MI CE и KEYBOARDS. Този избор елиминира риска от невалидни категории и осигурява типова защита; цената му е, че добавянето на нова категория изисква промяна в кода.

Голяма част от домейн обработката се извършва чрез binding модели за приемане на данни от форми – UserRegisterBindingModel, UserLoginBindingModel, ProfileUpdateBindingModel, AddCompanyBindingModel, ProductAddBindingModel, AddToCartBindingModel и CheckoutBindingModel. Те дефинират структурата на потребителския вход и концентрират валидационната логика чрез анотации като @Size, @NotBlank, @NotNull и

@Positive [10], намалявайки вероятността невалидни данни да достигнат до service слой. Освен entity и binding класове, проектът използва service модели за вътрешен трансфер и view модели за визуализация (UserServiceModel, CompanyViewModel, ProductViewModel, MaxPriceViewModel и др.). AutoMapper автоматизира стандартните трансформации, а смислово важните операции се извършват експлицитно в service слой.

2.3.3. Критични наблюдения и обобщение

Домейн моделът сравнително пряко следва основните потребителски сценарии: потребителят създава фирма; фирмата притежава продукти; продуктът се добавя в количката; количката участва във финализиране на поръчка; поръчката се архивира в историята. Анализът обаче позволява и няколко критични наблюдения: налице са дребни несъответствия в именуването (например колекцията в Company се казва companies, въпреки че съдържа продукти); поръчката е представена в силно опростен вид чрез исторически запис, без отделни order item обекти; някои валидационни правила са концентрирани само върху част от binding моделите; моделът не включва роли или права. Тези наблюдения очертават границите на текущата зрялост.

Проектирането на уеб-базираната платформа съчетава ясно дефинирани изисквания, добре подредена многослойна архитектура и компактен, но изразителен домейн модел. Тези три аспекта формират основата, върху която стъпва програмната реализация, разгледана в следващата глава.

3. ПРОГРАМНА РЕАЛИЗАЦИЯ

След представянето на изискванията, архитектурата и домейн модела следва анализ на конкретната програмна реализация. Тази глава разглежда последователно три аспекта: реализацията на back-end логиката, представящия слой и операционната среда – структурата на проекта, конфигурацията и build процеса.

3.1. Реализация на back-end логиката

Настоящият раздел стъпва директно върху наличния сорс код и показва как архитектурните идеи се материализират в програмен вид – чрез контролери, услуги, repository компоненти и конфигурация.

3.1.1. Инфраструктурна конфигурация и потребителски модул

Back-end поведението зависи от няколко централизирано дефинирани компонента, най-важни от които са `Mapper` и `PasswordEncoder`.

```
1 @Configuration
2 public class ApplicationBeanConfiguration {
3     @Bean
4     public Mapper mapper() {
5         return new Mapper();
6     }
7     @Bean
8     public PasswordEncoder passwordEncoder() {
9         return new Pbkdf2PasswordEncoder();
10    }
11 }
```

Listing 1: Конфигурация на mapper и password encoder

Листинг 1 илюстрира централизирано управление на зависимости: `Mapper` подпомага преобразуването между модели, а `Pbkdf2PasswordEncoder` осигурява сигурно съхранение на паролите.

Потребителският модул е най-обемен от гледна точка на уеб потоци и обхваща регистрация, потвърждение, вход, изход и профил. Регистрационният процес започва в `UserController`, където POST маршрутът приема `UserRegisterBindingModel` и при валидни данни делегира към `UserServiceImpl`.

```
1 @Override
```

```

2 public Optional<UserRegisterBindingModel> register(
3     UserRegisterBindingModel input) {
4     if (userRepository.findByEmail(input.getEmail()).isPresent()) {
5         return Optional.empty();
6     }
7     User user = new User();
8     user.setCompanies(new ArrayList<>());
9     user.setProductsCart(new ArrayList<>());
10    user.setHistoryProducts(new ArrayList<>());
11    user.setFirstName(input.getFirstName());
12    user.setLastName(input.getLastName());
13    user.setEmail(input.getEmail());
14    user.setConfirmed(false);
15    user.setPassword(passwordEncoder.encode(input.getPassword()));
16    user = userRepository.saveAndFlush(user);
17    SimpleMailMessage message = new SimpleMailMessage();
18    message.setTo(input.getEmail());
19    message.setSubject("Email Confirmation");
20    message.setText("http://localhost:8080/users/"
21        + user.getId() + "/confirm-email");
22    emailSender.send(message);
23    return Optional.of(input);
24 }

```

Listing 2: Опростен вид на регистрационната логика

Листинг 2 показва връзката между persistence, парола хеширане и e-mail интеграция, както и ограничения – URL за потвърждение е твърдо зададен към localhost:8080. Входът подава модела към услугата, която проверява паролата срещу хеша; при успех имейлът се записва в HttpSession. Логиката не използва Spring Security filter chain – достъпът до защитени ресурси се контролира ръчно чрез проверки за атрибут в сесията, което въвежда риск от дублиране на проверки. Потвърждението на акаунт се извършва чрез /users/{userId}/confirm-email, а профилният модул позволява редакция с проверка на сесията и на старата парола.

3.1.2. Модули за фирми, продукти и каталог

Управлението на фирмите чрез CompanyController и CompanyServiceImpl играе ролята на подготвителен етап за каталога: контролерът осигурява GET за визуализация и POST за запис чрез AddCompanyBindingModel, а услугата намира потребителя по имейл и създава entity Company. Фирмата принадлежи към конкретен потребител – системата се държи като персонален търговски workspace.

Продуктовият модул е разделен между ProductController, ProductsService и P

productServiceImpl. При добавяне на продукт услугата намира фирмата, създава Product, преобразува категорията чрез CategoryEnum.valueOf() и записва обекта. Каталогът е реализиран чрез /products/catalog/{category} – в контролера switch делегира към service методи, които филтрират продуктите по CategoryEnum. Решението е просто, но филтрирането върху цялата колекция е по-малко ефективно от целенасочена repository заявка. Клиентското филтриране по цена използва REST крайната точка за максимална цена.

За всеки продукт има детайлен екран за заявяване на количество. В ProductServiceImpl количеството на продукта се намалява и се добавя или обновява запис в количката: ако продуктът вече е там, съществуващият запис се увеличава; иначе се създава нов ShoppingCartEntity. Тук обаче се откроява архитектурна слабост: проверката е глобална върху всички записи, а не по текущ потребител, и сравнението е по име на продукта вместо по комбинация потребител-ID.

3.1.3. Количка, checkout и REST слой

Модулът за количка и checkout координира работа с потребител, количка и история. При отваряне на количката контролерът добавя списък с записи, брой продукти, обща цена и крайна цена с доставка; методът findAllShoppingCartEntities() връща всички записи без филтриране по потребител, което в многопотребителска среда не би било строго изолирано. Checkout процесът е реализиран в HistoryServiceImpl и е маркиран като @Transactional, тъй като променя множество състояния едновременно.

```
1 @Override
2 @Transactional
3 public void secureCheckout(String address, Object email,
4     Double totalPrice) throws Throwable {
5     User user = userRepository.findByEmail(email.toString())
6         .orElseThrow(() -> new Throwable("User not found"));
7     HistoryEntity historyEntity = new HistoryEntity();
8     historyEntity.setAddress(address);
9     historyEntity.setPrice(totalPrice);
10    historyEntity.setDate(LocalDate.now());
11    historyEntity.setUser(user);
12    historyRepository.save(historyEntity);
13    setNullToShoppingCartProducts(user.getId());
14    user.setProductsCart(new ArrayList<>());
15    user.getHistoryProducts().add(historyEntity);
16    userRepository.save(user);
17    shoppingCartRepository.deleteAllByUser_Id(user.getId());
18    deleteAllProductsWithZeroQuantity();
```

Listing 3: Същност на checkout операцията

Листинг 3 едновременно демонстрира как service слой координира сложна операция и опростения характер на модела: вместо реална поръчка с позиции се създава обобщен HistoryEntity, а продуктите с нулева наличност се изтриват изцяло. Освен HTML логиката, системата включва MaxPriceRestController (/api/products/max-price, връща MaxPriceViewModel), който показва, че същият домейн модел може да бъде ползван и за машинно четим интерфейс. Repository интерфейсите осигуряват кратко persistence ниво чрез методи като findByEmail, findAllByUser_Id и deleteAllByUser_Id.

Силните страни на back-end слоя включват последователно използване на service логика, отделяне на HTML и REST контролери, mapper и binding модели, password hashing, e-mail confirmation и транзакционност. Ограниченията са: ръчен сесиен контрол; непълна изолация по потребител; твърдо зададени стойности; изтриване на продукти с нулева наличност; опростена структура на поръчките.

3.2. Представящ слой и потребителско взаимодействие

Визуалният слой е реализиран чрез Thymeleaf шаблони с CSS, Bootstrap компоненти и ограничено количество JavaScript. Изборът е в съответствие с цялостната архитектура: интерфейсът е сървърно рендиран, което улеснява интеграцията между модели, валидация и HTML форми.

3.2.1. Организация на шаблоните и навигация

Шаблоните са разположени в src/main/resources/templates и отразяват основните екрани: index.html, login.html, register.html, profile.html, add-company.html, add-product.html, products-catalog.html, details.html, shopping-cart.html и history.html, заедно с допълнителни компоненти като users.html, companies.html, delivery.html и error шаблони. Те покриват целия потребителски цикъл. Визуалният слой не използва централен layout или fragment механизъм – всеки HTML файл е самостоятелен, което прави проекта лесен за локално проследяване, но води до дублиране на навигационната лента.

Таблица 4. Съответствие между шаблони и основни потребителски функции

Шаблон	Основна цел	Потребителски контекст
<code>index.html</code>	Начална страница	Гост
<code>register.html / login.html</code>	Регистрация и вход	Гост
<code>profile.html</code>	Преглед и редакция, списък с фирми	Удостоверен
<code>add-company.html / add-product.html</code>	Въвеждане на фирма/продукт	Удостоверен
<code>products-catalog.html</code>	Каталог и филтриране	Удостоверен
<code>details.html</code>	Избор на продукт и количество	Удостоверен
<code>shopping-cart.html</code>	Обобщение и checkout	Удостоверен
<code>history.html</code>	Преглед и изчистване на история	Удостоверен

Навигационната лента е най-видимият повтарящ се елемент. Тя присъства във всички екрани и осигурява постоянен достъп до основните действия, но повторението в множество шаблони създава риск от несъответствия при бъдещи промени. По-зрял подход би използвал Thymeleaf fragments. Допълнителен общ елемент е визуалният фон с фиксирано изображение и сходна цвятова схема, реализиран чрез inline стилове.

3.2.2. Thymeleaf, форми и каталог

Thymeleaf [6] играе централна роля: атрибути като `th:text`, `th:field`, `th:each`, `th:if`, `th:href` и `th:action` свързват директно подготвените от контролерите данни с HTML. Особено полезно е `th:field` при формите, което осигурява двупосочно свързване между binding модел и HTML полета и визуализиране на грешки чрез `#fields.hasErrors()`.

```

1 <form th:action="@{/products/add-product}"
2     th:object="{productAddBindingModel}"
3     th:method="POST">
4     <label for="name">Name</label>
5     <input th:field="*{name}" id="name" type="text" class="form-control">
6     <p th:if="{#fields.hasErrors('name')}}" class="errors alert alert-danger">
7         Name must be between 5 and 20 characters.
8     </p>
9 </form>

```

Listing 4: Типичен Thymeleaf фрагмент за форма

Почти всички съществени действия са реализирани чрез форми – регистрация, вход, профил, добавяне на фирма и продукт, количка и checkout. Те ползват Bootstrap класове като `form-group`, `form-control` и `btn`. При невалиден вход потребителят получава обратна връзка чрез `alert` елементи. Ограничение е, че съобщенията не са напълно стандартизирани – част от тях са твърдо кодирани в HTML файловете.

Каталогът комбинира навигация, странично меню с категории, ценови филтър и галерия от продуктови карти. Визуалният модел се опира на Bootstrap, но включва значително `inline CSS`. Всеки продукт се визуализира с изображение, име, категория и цена, а клиентският филтър по цена използва REST крайната точка. Страницата за детайли е по-концентрирана върху единичен обект и съдържа форма за количество и бутон за добавяне в количката. Екранът за количка разделя съдържанието на списък с избрани продукти и `summary` панел със суми, доставка, адрес и бутон за checkout – удачно UX решение, което показва едновременно детайла и обобщението. Профилният екран обединява редакция на лични данни и списък с фирми, а историята е реализирана с тъмен табличен интерфейс, разпознаваемо различен от останалите екрани.

Локализацията е реализирана чрез `messages.properties`, `messages_bg.properties` и `LocaleChangeInterceptor`. Връзки `?lang=en` и `?lang=bg` в навигацията позволяват смяна на езика, а голяма част от надписите използват ключове чрез `#{...}`. Налице са обаче и твърдо кодирани елементи, които не са синхронизирани между двата езика. Слойът използва Bootstrap, Font Awesome, jQuery и jQuery UI [15, 16] – в различни шаблони се ползват различни версии на Bootstrap, а ресурси са включени едновременно от локални файлове и CDN. UX силните страни включват видими действия в навигацията, отчетливи форми с грешки, разпознаваем каталог и обединен checkout екран. Ограниченията са: дублиране на навигационна логика; смесване на CDN, локални файлове и `inline CSS`;

частична локализация; отсъствие на централен layout; твърдо кодирани елементи.

3.3. Структура на проекта, конфигурация и експлоатационна среда

Освен архитектурата и домейн логиката, всяко завършено приложение следва да бъде разглеждано и от operational перспектива: организация в хранилището, зависимости, build процес, ресурси и генерирани артефакти.

3.3.1. Обща организация и зависимости

Проектът е структуриран като стандартно Maven приложение. В кореновата директория се намират `pom.xml`, Spring Boot wrapper файловете, `src` (src код), `target` (генерирани артефакти) и `doc` (LaTeX документацията). Това разделение позволява ясно отделяне на инженерния продукт от документирането.

Таблица 5. Ключови директории в проекта

Директория или файл	Предназначение
<code>pom.xml</code>	Зависимости, версия на Java, build конфигурация
<code>src/main/java</code>	Основен Java код
<code>src/main/resources/templates</code>	Thymeleaf шаблони
<code>src/main/resources/static</code>	CSS, JavaScript, изображения
<code>src/main/resources/application.properties</code>	Локална конфигурация (DB, mail, JPA)
<code>target/classes</code>	Компилирани класове и ресурси
<code>doc</code>	LaTeX документация и библиография

Файлът `pom.xml` декларира родителския Spring Boot артефакт, Java версията и списъка със зависимости. Системата използва типичен набор от библиотеки за Spring уеб

приложение, обобщени в таблица 6.

Таблица 6. Основни зависимости и роля в проекта

Зависимост	Функция
spring-boot-starter-web	MVC и HTTP слой [4, 5]
spring-boot-starter-thymeleaf	Сървърно рендиране шаблони [6]
spring-boot-starter-data-jpa	Persistence чрез JPA [7]
mysql-connector-java	JDBC драйвер за MySQL [17]
spring-boot-starter-validation	Валидация на binding моделите [10]
modelmapper	Преобразуване между модели [18]
spring-context-support, javax.mail	E-mail интеграция [19]
spring-security-crypto	Хеширане на пароли [11]
spring-boot-starter-test	Тестова инфраструктура

Анализът на `pom.xml` разкрива и важни наблюдения: `spring-security-crypto` присъства два пъти с различни версии; `spring-boot-starter-data-jpa` е декларирана с LATEST, което не е добра практика; някои security зависимости са коментирани, което подсказва изоставена първоначална идея за по-богата интеграция.

3.3.2. Версии, конфигурация и build процес

Проектът използва Spring Boot 2.6.2 и Java 11 – разумна, дългосрочно поддържана комбинация. Операционното поведение се управлява от `application.properties`, който задава логване, MySQL връзка, JPA и SMTP конфигурация. Базата `onlineShop` се създава

автоматично при нужда, а Hibernate е настроен с `ddl-auto=update`. Използването на `spring.jpa.open-in-view=false` е добра практика, която намалява вероятността от неочаквани достъпи до базата по време на визуализация. SMTP настройките (`spring.mail.host`, порт, TLS, автентикация) позволяват изпращане на потвърждаващи имейли. Тук обаче се вижда важно operational ограничение – чувствителни данни са записани директно в конфигурационния файл; за production следва да се използват environment variables или секретно хранилище.

Maven [20] осигурява централизирано управление на зависимости, възпроизводим build и интеграция със Spring Boot Maven plugin. Build процесът попълва `target/classes` с `.class` файловете и копираните ресурси от `src/main/resources`. За локално стартиране са необходими Java 11, работещ MySQL сървър, валидни данни за достъп, e-mail конфигурация и Maven (или включеният wrapper). Operational рисковете включват чувствителни параметри в изходния код, LATEST в зависимост, дублирани зависимости, липса на разграничени профили и отсъствие на миграции чрез Flyway или Liquibase – естествени посоки за бъдеща хигиенизация.

Програмната реализация демонстрира ясно приложени принципи на Spring базирана уеб система: контролери за HTTP потока, service слой за домейн логика, repository интерфейси за persistence, mapper за трансформации и инфраструктурна конфигурация. Структурата на проекта, Maven конфигурацията и operational средата осигуряват добра основа за стартиране и анализ, а конкретните области за подобрене – сигурност, изолация по потребител, централизация на front-end и управление на чувствителна конфигурация – очертават насоки за развитие.

4. РЪКОВОДСТВО ЗА ИЗПОЛЗВАНЕ

Тази глава представя ръководство за използване на системата чрез поредица потребителски сценарии, последвани от верификация на реализираната функционалност. Първо се проследяват основните демонстрационни сценарии, които обхващат целия жизнен цикъл на работа със системата. След това се представят използваният подход към верификацията и матрицата за функционална верификация, които потвърждават, че описаните сценарии имат реална програмна основа.

4.1. Потребителски сценарии

Една от важните функции на техническата дипломна документация е да направи връзка между архитектурното описание и реалната употреба на системата. Поради тази причина в настоящия раздел са представени основните потребителски сценарии в последователен демонстрационен вид. Всеки сценарий е придружен от описание на очакваното поведение на системата и от предварително предвидено място за screenshot от работещото приложение.

Този подход има две предимства. От една страна, текстът действа като кратко ръководство за демонстрация. От друга страна, структурата позволява реални екранни снимки да бъдат добавени на по-късен етап без нужда от пренаписване на документа. Така дипломната работа остава едновременно съдържателно завършена и practically подготвена за визуално допълване.

4.1.1. Сценарий 1: Начална страница и избор на език

Демонстрацията започва от началната страница на приложението. Тя има ролята на публична входна точка и предоставя достъп до регистрация, вход и избор на език. При липса на активна сесия системата не позволява използване на вътрешните функции, а се държи като портал към удостоверителния процес. Визуално началната страница съдържа хедър, навигационна лента, hero секция и основни бутони за навигация.

От потребителска гледна точка това е правилен първи екран, защото поставя акцент върху най-важните начални действия: регистриране или вход. Паралелно с това наличието на връзки за EN и BG в навигацията демонстрира, че приложението поддържа базова локализация още от първия момент на взаимодействие.

Placeholder за screenshot

Начална страница и езиков превключвател

Тук да се постави screenshot на началната страница, на който ясно да се виждат навигационната лента, hero секцията и връзките за смяна на езика.

Фигура 4. Начална страница и езиков превключвател

4.1.2. Сценарий 2: Регистрация на нов потребител

Следващата естествена стъпка е регистрацията. Потребителят отваря формата за регистрация и попълва име, фамилия, имейл и парола. При коректно подадени данни системата създава нов профил и инициира изпращане на потвърждаващ e-mail. Ако входът не е коректен или ако вече съществува потребител със същия имейл, се визуализира подходяща обратна връзка.

Този сценарий има ключова демонстрационна стойност, защото показва едновременно работа с форма, server-side обработка, persistence операция и интеграция с външен mail компонент. Именно тук се вижда, че проектът надхвърля чисто визуален прототип и реализира реален поток по създаване на акаунт.

Placeholder за screenshot

Форма за регистрация

Тук да се постави screenshot на екрана за регистрация с попълнени примерни данни или с демонстрация на валидационни съобщения при невалиден вход.

Фигура 5. Форма за регистрация

4.1.3. Сценарий 3: Потвърждение на акаунт и вход

След изпращане на регистрационната форма потребителят следва да потвърди своя акаунт чрез връзката, получена по електронна поща. В практическа демонстрация може да се покаже както самият имейл, така и резултатът от активиране на линка – преминаване към форма за вход. След потвърждение потребителят въвежда имейл и парола, а системата проверява валидността на данните и създава активна сесия.

От гледна точка на потребителската история това е преходът от публичната към вътрешната част на приложението. От този момент нататък системата започва да показва допълнителни навигационни елементи, които са достъпни само при активна сесия.

Placeholder за screenshot

Вход в системата след потвърдена регистрация

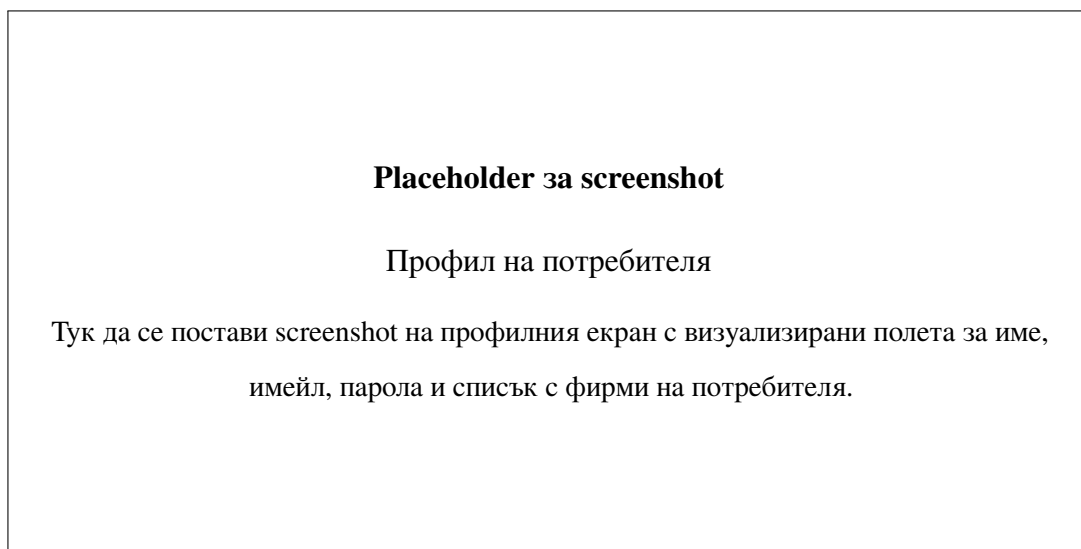
Тук да се постави screenshot на login формата или на резултата след успешно удостоверяване, например пренасочване към каталога.

Фигура 6. Вход в системата след потвърдена регистрация

4.1.4. Сценарий 4: Профил на потребителя и списък с фирми

След вход един от най-важните екрани е профилът. Той позволява на потребителя да прегледа своите основни данни и при необходимост да ги редактира. Допълнително в страничната част на екрана се визуализира списък от свързани фирми и е наличен бутон за добавяне на нова.

Този екран е показателен за комбиниран характер на интерфейса: той не е само форма за редакция, а едновременно работна зона, която обединява идентичност на потребителя и свързана бизнес информация. При демонстрация е подходящо да се покаже както нормалното състояние на профила, така и пример за съобщение при неправилно въведена стара парола или несъвпадение между новата и потвърдената парола.



Фигура 7. Профил на потребителя

4.1.5. Сценарий 5: Добавяне на фирма

Формата за добавяне на фирма е логически подготвителна стъпка за последващото въвеждане на продукти. Потребителят посочва име на фирмата, идентификационни данни, адрес и тип на регистрация. След успешен запис фирмата става достъпна в списъка от фирми и може да бъде избрана от формата за нов продукт.

Този сценарий е особено полезен за демонстрация пред комисия, защото показва, че приложението не работи с изолирани продукти, а поддържа по-реалистична домейн структура, в която продуктът се асоциира с фирма. По този начин каталогът се обвързва с източник на данни, а не с анонимни артикули.

Placeholder за screenshot

Форма за добавяне на фирма

Тук да се постави screenshot на екрана за добавяне на фирма, включително полетата за име, MOL, VAT, адрес и статус на регистрация.

Фигура 8. Форма за добавяне на фирма

4.1.6. Сценарий 6: Добавяне на продукт

След като съществува поне една фирма, потребителят може да премине към добавяне на продукт. Екранът за това включва полета за име, URL към изображение, цена, количество, категория и избор на фирма. Категорията се избира от предварително дефиниран enum списък, а фирмата – от асоциираните с потребителя компании.

Този сценарий показва няколко важни характеристики на приложението. Първо, демонстрира комбиниране на текстови, числови и избираеми полета. Второ, показва как входните данни се валидират още на ниво форма. Трето, илюстрира как домейн връзките вече се материализират в UI: потребителят не въвежда фирма на свободен текст, а я избира от наличен набор.

Placeholder за screenshot

Добавяне на нов продукт

Тук да се постави screenshot на формата за добавяне на продукт, на който ясно да се виждат изборът на категория, фирма и валидационните полета.

Фигура 9. Добавяне на нов продукт

4.1.7. Сценарий 7: Каталог и филтриране по категория и цена

След добавяне на продукти системата може да бъде демонстрирана чрез каталога. Това е един от най-визуалните екрани в приложението. Потребителят вижда странично меню с категории и зона с продуктови карти. Допълнително JavaScript логиката използва REST крайната точка за максимална цена, за да конфигурира ценови плъзгач за клиентско филтриране.

При демонстрация е подходящо да се покажат поне три действия: отваряне на общия каталог, преминаване към конкретна категория и стесняване на видимите продукти чрез ценовия диапазон. Този сценарий представя най-добре съчетанието между сървърно рендиран интерфейс и ограничена клиентска динамика.

Placeholder за screenshot

Каталог с продукти

Тук да се постави screenshot на каталога с няколко продукта, страничното меню с категории и активния ценови филтър.

Фигура 10. Каталог с продукти

4.1.8. Сценарий 8: Детайли на продукт и добавяне в количка

След избор на конкретен артикул потребителят преминава към страницата с детайли. Там се визуализират изображение, наименование, цена и текуща наличност. Потребителят въвежда желаното количество и го добавя в количката. Ако количеството е невалидно или надвишава наличността, системата връща съобщение за грешка.

Този екран има съществена демонстрационна роля, защото именно тук се вижда преходът от пасивно разглеждане към активна транзакционна интеракция. От UX гледна точка формата е минималистична и фокусира вниманието върху единичния продукт.

Placeholder за screenshot

Детайлен изглед на продукт

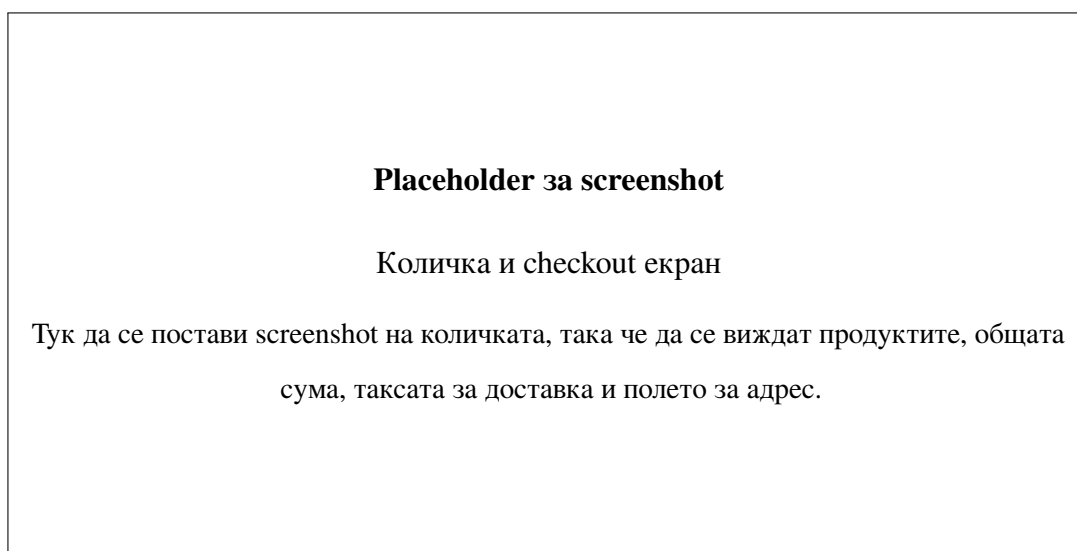
Тук да се постави screenshot на страницата с детайли на продукт, включително полето за количество и бутона за добавяне в количката.

Фигура 11. Детайлен изглед на продукт

4.1.9. Сценарий 9: Количка и финализиране на поръчка

След добавяне на продукти в количката потребителят отваря съответния екран, където вижда избраните артикули, броя им, цената на всеки запис, общата стойност, таксата за доставка и крайна сума. В summary панела се попълва адресът, след което се извършва checkout.

Този сценарий е особено важен, защото демонстрира, че приложението не спира до каталог и визуализация, а реализира реален край на потребителската цел. При защита на дипломна работа именно тази страница често е сред най-силните екрани за демонстрация, защото интегрира back-end изчисления, визуално обобщение и form-based завършване на операцията.



Фигура 12. Количка и checkout екран

4.1.10. Сценарий 10: Екран за успешна поръчка и история

След завършване на покупката потребителят се пренасочва към потвърждаващ екран, а впоследствие може да отвори своята история на поръчките. Историческият изглед показва списък с реализираните покупки, включително идентификатор, имейл, адрес, дата и цена. Наличен е и бутон за изчистване на историята.

Този етап е важен за демонстрацията, защото визуализира персистентния резултат от целия потребителски процес. Докато каталожните и checkout екраните показват текущо състояние, историята показва, че системата поддържа и времева следа за вече реализирани действия.

Placeholder за screenshot

История на поръчките

Тук да се постави screenshot на екрана с история на поръчките, включително таблицата с дата, адрес и цена на направените покупки.

Фигура 13. История на поръчките

4.1.11. Сценарий 11: Демонстрация на локализацията

Допълнителен, но важен демонстрационен сценарий е смяната на езика. Тъй като системата поддържа базова английска и българска локализация, е полезно в дипломната работа да се покаже поне един и същи екран в двата режима. Това дава визуално доказателство, че локализационната конфигурация не е само декларативно налична, а реално участва в поведението на приложението.

При добра демонстрация могат да бъдат сравнени например началната страница, формата за добавяне на продукт или екранът за профил в двата езикови режима. Подобна съпоставка е особено ценна, защото показва и ограниченията на текущата локализация – къде тя е пълна и къде остават твърдо кодирани текстове.

Placeholder за screenshot

Съпоставка между английски и български интерфейс

Тук да се постави двойка screenshots или комбиниран кадър, показващ един и същ екран на английски и на български език.

Фигура 14. Съпоставка между английски и български интерфейс

4.2. Верификация на функционалността

След представянето на потребителските сценарии следва да се покаже, че те имат реална програмна опора в кода на приложението. Целта на този раздел не е просто да се повтори, че системата съдържа дадена функционалност, а да се покаже доколко решението удовлетворява поставените изисквания и какви са доказателствата за неговото съществуване.

В настоящия случай верификацията е извършена предимно чрез аналитичен подход върху реалния код, маршрути, шаблони, конфигурация и взаимовръзки между модулите. Това означава, че проверката се базира на действително реализираната логика на приложението, а не само на намерения или проектни декларации. Тъй като проектът има учебно-приложен характер и липсва богата автоматизирана тестова база, именно този тип инженерно-аналитична верификация е най-подходящ за дипломната работа.

4.2.1. Подход към верификацията

За целите на настоящата верификация са използвани няколко взаимнодопълващи се подхода.

Статичен анализ на сорс кода Първият подход е проследяване на действителната логика в контролерите, услугите, repository интерфейсите, entity моделите и шаблоните. Чрез него може да бъде установено дали отделните потребителски сценарии имат реална

програмна опора, каква е последователността на операциите и къде са разположени ключовите проверки за валидност, достъп или коректност на данните.

Сценарийна проверка на функционалността Вторият подход се базира на дефинираните по-горе потребителски сценарии. За всеки от тях се проверява дали в системата има наличен контролерен маршрут, service логика, подходящ шаблон и persistence поведение, които заедно реализират целевия резултат. По този начин се формира сценарийна матрица на верификация.

Архитектурна и качествена оценка Третият подход е по-качествен и разглежда не само наличието на дадена функция, а начина, по който тя е реализирана. Тук се оценяват свойства като структурираност на кода, повторна използваемост, разширяемост, защита на чувствителни данни, consistency на интерфейса и operational дисциплина.

Съвкупността от тези три перспективи дава достатъчно убедителна рамка за верификация на проекта.

4.2.2. Матрица за функционална верификация

В таблица 7 е представена обобщена матрица за функционална верификация. Тя не замества автоматизираното тестване, но систематизира дали ключовите функции са реално представени в кода и какви са доказателствата за тяхното съществуване.

Таблица 7. Матрица за функционална верификация

Сценарий	Проверявана функционалност	Основа за верификация	Извод
V-1	Регистрация на потребител	Налични /users/register GET/POST маршрути, service логика за запис и mail изпращане	Функцията е реализирана последователно и включва persistence и e-mail стъпка
V-2	Потвърждение на акаунт	Наличен маршрут /users/{userId}/confirm-email и service метод confirm	Потвърждението е реализирано в опростен, но работещ вид

Сценарий	Проверявана функционалност	Основа за верификация	Извод
V-3	Вход и изход	Налични login маршрути, проверка на парола и invalidate на сесията	Функцията е налице, но е базирана на ръчно сесийно управление
V-4	Редакция на профил	Налични profile маршрути, проверки на стара парола и matching на нови пароли	Функцията е реализирана с базова входна защита
V-5	Добавяне на фирма	CompanyController, AddCompanyBindingModel, CompanyServiceImpl	Реализацията е ясна и логически свързана с потребителя
V-6	Добавяне на продукт	ProductController, ProductAddBindingModel, ProductServiceImpl	Налични са всички необходими елементи за запис на продукт
V-7	Каталог по категории	Категорийни маршрути, шаблон products-catalog.html, enum категории	Функцията е реализирана, макар и с филтриране на ниво service вместо repository
V-8	Детайлен преглед и добавяне в количка	Маршрут /products/{id}/details, проверка на количество, update на количката	Реализацията е налична, но изолацията по потребител е непълна
V-9	Checkout	shopping-cart POST маршрут и HistoryServiceImpl.secureCheckout	Функцията е реализирана чрез транзакционна координация на няколко операции
V-10	История на поръчките	/products/my-history GET/POST маршрути и HistoryRepository	Налице е основен исторически модул с възможност за изчистване
V-11	Локализация	messages.properties, messages_bg.properties, I18nConfig	Поддръжката е реална, но частично непълна

Сценарий	Проверявана функционалност	Основа за верификация	Извод
V-12	REST крайна точка	MaxPriceRestController и service метод за максимална цена	Налице е примерна API функционалност

Матрицата показва, че всички основни функции, формулирани в изискванията, имат конкретни програмни реализации. Това е съществен резултат, защото доказва, че системата не е само архитектурно упражнение, а реален функционален софтуерен продукт. В същото време няколко от сценариите ясно подсказват и ограничителни фактори, които ще бъдат разгледани в Заключение.

ЗАКЛЮЧЕНИЕ

В рамките на настоящата дипломна работа беше проектирана и разработена уеб базирана система за електронна търговия с компютърна и офис техника. Системата е реализирана като Java Spring Boot приложение и покрива основните процеси на компактен онлайн магазин: регистрация, потвърждение на акаунт, вход, поддръжка на профил, управление на фирми и продукти, разглеждане на каталог, работа с количка, финализиране на поръчка и история на покупки. Архитектурата е организирана в ясно разграничени слоеве – контролери, услуги, repository интерфейси, entity модели и Thymeleaf шаблони, което осигурява добра проследимост и поддръжаемост.

Изборът на Spring Boot, Spring MVC, Thymeleaf, Spring Data JPA и MySQL е напълно оправдан за подобна задача. Hibernate Validator, ModelMapper, JavaMail интеграция и Pbkdf2PasswordEncoder допълнително повишават качеството на решението. Архитектурната подреденост, поддръжаемостта и разширяемостта са сред най-силните страни на проекта. Идентифицирани са и конкретни ограничения: достъпът се управлява чрез HttpSession без централизирана security рамка; моделът на поръчките е опростен чрез HistoryEntity без отделни позиции и статуси; чувствителна конфигурация е в application.properties; локализацията и front-end организацията са частично непоследователни.

Естествените насоки за бъдещо развитие са: с висок приоритет – въвеждане на Spring Security, преработване на модела за поръчки чрез Order и OrderItem и изнасяне на чувствителна конфигурация във външно хранилище; със среден приоритет – централизация на front-end чрез Thymeleaf fragments, разширяване на локализацията, автоматизирани тестове и миграции на схемата (Flyway или Liquibase); по-нататък – разширени търговски функции, контейнеризация и CI/CD pipeline.

Поставената цел е постигната: създадена е работеща, концептуално ясна и надграждаема система, описана в академично издържана форма и оценена както през призмата на постигнатото, така и на нейните ограничения. Това прави проекта убедителен завършек на дипломна разработка и стабилна основа за бъдещо инженерно надграждане.

ИЗПОЛЗВАНА ЛИТЕРАТУРА

- [1] Shopify Inc., “Shopify help center and merchant documentation.” <https://help.shopify.com/en/manual>, 2026. Official documentation, accessed 2026-04-07.
- [2] Automattic, “Woocommerce documentation.” <https://woocommerce.com/documentation/>, 2026. Official documentation, accessed 2026-04-07.
- [3] Adobe Inc., “Adobe commerce and magento open source documentation.” <https://developer.adobe.com/commerce/docs/>, 2026. Official documentation, accessed 2026-04-07.
- [4] VMware, Inc., “Spring boot reference documentation, version 2.6.2.” <https://docs.spring.io/spring-boot/docs/2.6.2/reference/htmlsingle/>, 2026. Official documentation, accessed 2026-04-07.
- [5] VMware, Inc., “Spring framework reference documentation: Web mvc.” <https://docs.spring.io/spring-framework/docs/5.3.15/reference/html/web.html>, 2026. Official documentation, accessed 2026-04-07.
- [6] Thymeleaf Team, “Using thymeleaf.” <https://www.thymeleaf.org/doc/tutorials/3.0/usingthymeleaf.html>, 2026. Official documentation, accessed 2026-04-07.
- [7] VMware, Inc., “Spring data jpa reference documentation.” <https://docs.spring.io/spring-data/jpa/docs/2.6.0/reference/html/>, 2026. Official documentation, accessed 2026-04-07.
- [8] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [9] I. Sommerville, *Software Engineering*. Pearson, 10th ed., 2015.
- [10] Hibernate.org, “Hibernate validator reference guide.” https://docs.jboss.org/hibernate/validator/reference/en-US/html_single/, 2026. Official documentation, accessed 2026-04-07.
- [11] VMware, Inc., “Spring security reference: Password storage.” <https://docs.spring.io/spring-security/reference/features/authentication/password-storage.html>, 2026. Official documentation, accessed 2026-04-07.

- [12] OWASP Foundation, “Password storage cheat sheet.” https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html, 2026. Security guidance, accessed 2026-04-07.
- [13] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [14] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.
- [15] Bootstrap Authors, “Bootstrap documentation, version 5.1.” <https://getbootstrap.com/docs/5.1/getting-started/introduction/>, 2026. Official documentation, accessed 2026-04-07.
- [16] OpenJS Foundation, “jQuery api documentation.” <https://api.jquery.com/>, 2026. Official documentation, accessed 2026-04-07.
- [17] Oracle Corporation, “Mysql 8.0 reference manual.” <https://dev.mysql.com/doc/refman/8.0/en/>, 2026. Official documentation, accessed 2026-04-07.
- [18] ModelMapper Contributors, “Modelmapper user manual.” <https://modelmapper.org/user-manual/>, 2026. Official documentation, accessed 2026-04-07.
- [19] VMware, Inc., “Spring framework reference documentation: Email integration.” <https://docs.spring.io/spring-framework/reference/integration/email.html>, 2026. Official documentation, accessed 2026-04-07.
- [20] Apache Software Foundation, “Apache maven project documentation.” <https://maven.apache.org/guides/>, 2026. Official documentation, accessed 2026-04-07.